



Inżynieria Oprogramowania

dr Katarzyna Grzesiak-Kopeć





01

Miary oprogramowania

Pomiar, miara, metryka



Pomiar: czynność

Miara: liczba, wymiar, pojemność, rozmiar cechy

Metryka: ilościowa miara stopnia w jakim system, komponent lub proces posiada dany atrybut

Po co mierzyć?

- ✓ Określenie jakości produktu lub procesu
- ✓ Przewidzenie jakości produktu lub procesu
- ✓ Poprawa jakości produktu lub procesu



You can neither predict nor control
what you cannot measure.



Tom DeMarco

Po co mierzyć?

- ✓ Kontrola
- ✓ Zrozumienie
- ✓ Porównanie
- ✓ Predykcja



Tom DeMarco

UDOSKONALANIE

Dobra metryka

- ✓ Prosta i łatwa do obliczenia
- ✓ Przekonująca empirycznie i intuicyjnie
- ✓ Spójna i obiektywna
- ✓ Zgodna z rzeczywistymi jednostkami i wymiarami
- ✓ Niezależna od języka programowania
- ✓ Pomocna w dokonaniu oceny jakości



Zalety stosowania metryk



- ✓ Znane kryteria sukcesu i porażki
produktu, procesu lub działalności ludzkiej
- ✓ Można ocenić efektywność wprowadzonych zmian
produktu, procesu lub działalności ludzkiej
- ✓ Ułatwione podejmowanie decyzji technicznych oraz kierowniczych
- ✓ Rozpoznawanie trendów i dokonywanie estymacji

Co mierzymy?



PRODUKTY

Rezultaty powstałe w czasie tworzenia oprogramowania
Kod źródłowy, specyfikacja, plan testów, dokumentacja...

PROCESY

Czynności wykonywane w trakcie tworzenia oprogramowania
Projektowanie, kodowanie, ...

ZASOBY

Zasoby wykorzystywane w trakcie tworzenia oprogramowania
Sprzęt, wiedza, metody, ludzie, ...

Jakie atrybuty mierzymy?



WEWNĘTRZNE

- > Mierzone są same elementy
- > Rozmiar, czas, cena, ...

ZEWNĘTRZNE

- > Mierzone w stosunku do środowiska
- > Niezawodność, wydajność, skalowalność...

ELEMENT	ATRYBUT
Projekt	Liczba błędów wykrytych w trakcie inspekcji
Specyfikacja projektowa	Liczba stron
Kod źródłowy	Liczba linii kodu, liczba operacji
Zespół deweloperów	Rozmiar zespołu, przeciętne doświadczenie

Rodzaje metryk



MIARY BEZPOŚREDNIE

Koszt i liczba pracowników w projekcie

Liczba wierszy kodu źródłowego (LOC)

MIARY POŚREDNIE

Defect density = liczba błędów / rozmiar

PROGNOZOWANIE

NP. Prognoza wielkości systemu na podstawie pomiaru funkcjonalności

Rodzaje metryk

NOMINALNE

3GL, 4GL

PORZĄDKOWE

Wydajność: słaba, przeciętna, wysoka

PRZEDZIAŁOWE

1-150 LOC/day

WZGLĘDNE

Produkt 2x większy od poprzedniego

BEZWZGLĘDNE

500 000 LOC



Produkt vs proces

METRYKI PROCESU

- > Analiza paradygmatów, zadań, punktów milowych
- > Prowadzi do udoskonalania procesu – udoskonalanie długoterminowe

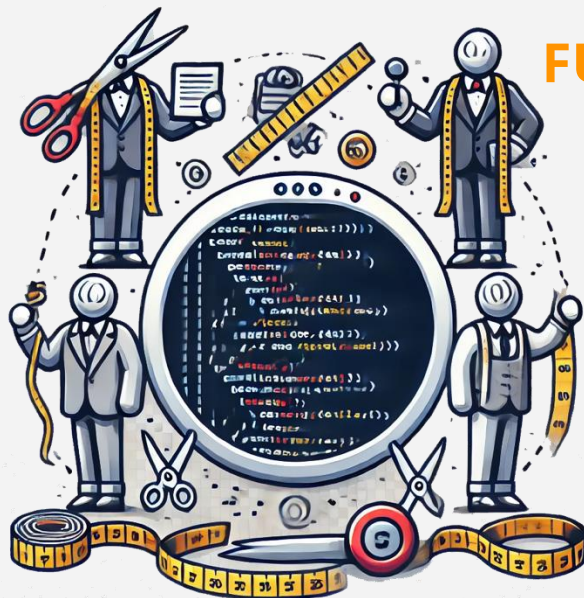


METRYKI PRODUKTU

- > Ocena stanu projektu
- > Uchwycenie potencjalnego ryzyka
- > Identyfikacja problemów
- > Dostosowanie "workflowu" oraz/lub zadań
- > Ocena umiejętności zespołu by zapewnić odpowiednią jakość produktu

Metryki produktu

ROZMIAR



FUNKcjONALNOŚĆ

ZŁOŻONOŚĆ

• Metryki produktu: ROZMIAR •

Lines of Code LOC

Linia/wiersz kodu, Conte 1986

• Metryki produktu: ROZMIAR •

LOC

Linia/wiersz kodu, Conte 1986

Linia kodu nazywamy każdą linię tekstu napisanego programu, która nie jest ani komentarzem ani linią pustą, bez względu na liczbę instrukcji w linii. W szczególności są to linie zawierające nagłówki, deklaracje, wykonywalne i niewykonywalne instrukcje programu.

• Metryki produktu: ROZMIAR •

LOC



Rozmiar kodu źródłowego



LOC, 1000 Lines Of Code (KLOC)



Liczba osobomiesięcy



Defects/KLOC



Cost/LOC



Documentation Pages/KLOC

Zalety

Wady

ŁATWA DO OBLICZENIA	ZALEŻY OD JĘZYKA PROGRAMOWANIA
DOBRZE ZNANA I OPISNA	NIE PROMUJE KRÓTKICH, DOBRZE NAPISANYCH PROGRAMÓW
JEST BAZĄ DLA WIELU MODELI PROGNOSTYCZNYCH	TRUDNO UŻYĆ DO PROGNOZY? JAK POLICZYĆ NA ETAPIE PLANOWANIA?

Metryki produktu: FUNKCJONALNOŚĆ

Function Point Analysis (FPA)



International Function Point Users Group, www.ifpug.org



IFPUG
International **Function** Point
USERS GROUP

Meaningful Metrics: Developed in the late 1970s by Allan Albrecht as a means of measuring software size in terms that are meaningful to end users, function points have become an international standard used to measure software size (ISO 20926:2009).

Metryki produktu: FUNKCJONALNOŚĆ

Function Point Analysis (FPA)



International Function Point Users Group, www.ifpug.org



Miara pośrednia



Wyliczana na podstawie doświadczalnie dobranego wzoru na podstawie liczby funkcji:

wejście (input), wyjście (output), zapytania (queries), pliki

wewnętrzne

(internal files), pliki zewnętrzne (external interfaces)

• Metryki produktu: **FUNKcjONALNOŚĆ** •

Function Point Analysis (FPA)

01 wejście (input): wprowadza dane, nie są to zapytania

02 wyjście (output): raporty, okienka, komunikaty... liczba raportów,
nie elementów

03 zapytania (queries): pytanie i natychmiastowa odpowiedź, każde
zapytanie liczy się oddzielnie

04 pliki wewnętrzne (internal files): różne dane liczymy oddzielnie

• 05 pliki zewnętrzne (external interfaces): zewnętrzne interfejsy •

Metryki produktu: FUNKCJONALNOŚĆ

			Wagi				
Parametr	Liczba		Proste	Średnie	Złożone		
Wejście		X	3	4	6	=	
Wyjście		X	4	5	7	=	
Zapytanie		X	3	4	6	=	
Pliki		X	7	10	15	=	
Interfejsy		X	5	7	10	=	
Suma (S)							

Metryki produktu: FUNKCJONALNOŚĆ

			Wagi				
Parametr	Liczba		Proste	Średnie	Złożone		
Wejście		X	3	4	6	=	
Wyjście							
Zapytanie							
Pliki							
Interfejsy							
Suma (S)							

$$PF = S \times [0.65 + 0.01 \times \sum(F_i)]$$

Metryki produktu: FUNKCJONALNOŚĆ

$$PF = S \times [0.65 + 0.01 \times \sum(F_i)]$$

- ✓ Współczynniki F_i ustalany na podstawie odpowiedzi na serię pytań
 - ✓ Każda odpowiedź jest liczbą 0 (nieistotne) do 5 (absolutnie kluczowe)
- (1) Czy system wymaga tworzenia backupów i odzyskiwania danych?
- (3) Czy dane przetwarzane są w sposób rozproszony?
- (10) Czy algorytmy przetwarzania danych są bardzo złożone?
- (12) Czy projekt obejmuje wdrożenie?
- (14) Czy użytkownik ma mieć możliwość modyfikowania systemu. Czy łatwość użytkowania jest ważna?

• Metryki produktu: FUNKCJONALNOŚĆ •

Software Economics and Function Point Metrics:

Thirty years of IFPUG Progress

Version 10.0

April 14, 2017



Metryki produktu: FUNKCJONALNOŚĆ

Software Economics and Function Point Metrics:

Thirty years of IFPUG Progress

Languages	Size in	Total	Work hours	FP per	Work	Work hours	LOC per
	KLOC	Work hours	per FP	Month	Months	per KLOC	Month
1 Machine language	640.00	119,364	119.36	1.11	904.27	186.51	708
2 Basic Assembly	320.00	61,182	61.18	2.16	463.50	191.19	690
3 JCL	220.69	43,125	43.13	3.06	326.71	195.41	675
4 Macro Assembly	213.33	41,788	41.79	3.16	316.57	195.88	674
5 HTML	160.00	32,091	32.09	4.11	243.11	200.57	658
6 C	128.00	26,273	26.27	5.02	199.04	205.26	643
7 XML	128.00	26,273	26.27	5.02	199.04	205.26	643

Metryki produktu: FUNKCJONALNOŚĆ

Software Economics and Function Point Metrics:

Thirty years of IFPUG Progress

Languages	Size in KLOC	Total Work hours	Work hours per FP	FP per Month	Work Months	Work hours per KLOC	LOC per Month
31 C++	53.33	12,697	12.70	10.40	96.19	238.07	554
32 Go	53.33	12,697	12.70	10.40	96.19	238.07	554
33 Java	53.33	12,697	12.70	10.40	96.19	238.07	554
34 PHP	53.33	12,697	12.70	10.40	96.19	238.07	554
35 Python	53.33	12,697	12.70	10.40	96.19	238.07	554
36 C#	51.20	12,309	12.31	10.72	93.25	240.41	549
6 C	128.00	26,273	26.27	5.02	199.04	205.26	643
7 XML	128.00	26,273	26.27	5.02	199.04	205.26	643

Metryki produktu: FUNKCJONALNOŚĆ

Software Economics and Function Point Metrics:

Thirty years of IFPUG Progress

	Languages	Size in	Total	Work hours	FP per	Work	Work hours per	LOC per
72	TranscriptSQL	12.80	5,327	5.33	24.78	40.36	416.19	317
73	QBE	12.80	5,327	5.33	24.78	40.36	416.19	317
74	X	12.80	5,327	5.33	24.78	40.36	416.19	317
75	Mathematica10	9.14	4,662	4.66	28.31	35.32	509.94	259
76	BPM	7.11	4,293	4.29	30.75	32.52	603.69	219
77	Generators	7.11	4,293	4.29	30.75	32.52	603.69	219
78	Excel	6.40	4,164	4.16	31.70	31.54	650.57	203
79	IntegraNova	5.33	3,970	3.97	33.25	30.07	744.32	177
7	XML	128.00	26,273	26.27	5.02	199.04	205.26	643

Metryki produktu: **FUNKCJONALNOŚĆ**

Function Point Analysis (FPA) użycie:



Ocena złożoności realizacji systemów



Audyt projektów



Wybór systemów informatycznych funkcjonujących w przedsiębiorstwie do reinżynierii (koszt utrzymania/FPs)



Szacowanie liczby testów



Ocena jakości pracy i wydajności zespołów ludzkich



IFPUG
International **Function** Point
USERS GROUP

Metryki produktu: FUNKCJONALNOŚĆ

Function Point Analysis (FPA) użycie:



IFPUG
International **Function** Point
USERS GROUP



Ocena stopnia zmian, wprowadzanych przez użytkownika

na poszczególnych etapach budowy systemu

Prognozowanie

kosztów  pielęgnacji i rozwoju systemów

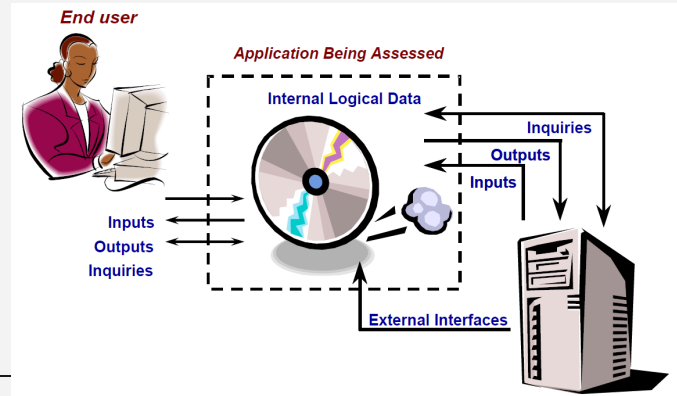


Porównanie i ocena różnych ofert oprogramowania pod kątem merytorycznym i kosztowym

Metryki produktu: FUNKCJONALNOŚĆ

ZALETY

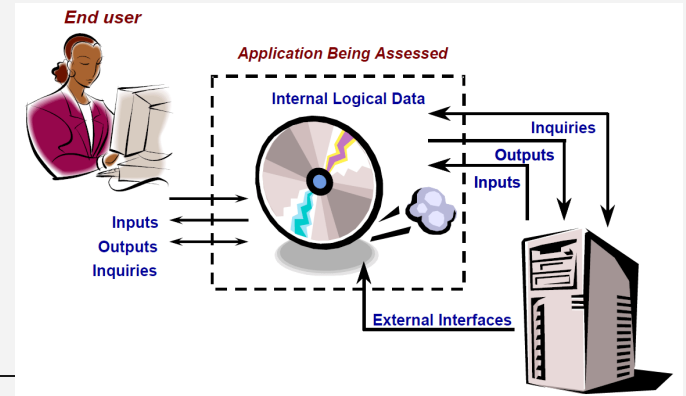
- > Pomaga w zarządzaniu wymaganiami; rozmiar funkcjonalny
- > Umożliwia normalizację metryk w celu zapewnienia spójnej analizy
- > Podstawa oszacowania kosztu i harmonogramu
- > Niezależna od technologii, platformy i języka
- > Udokumentowana i powtarzalna
- > Pozwala ilościowo oceniać funkcjonalność



Metryki produktu: FUNKCJONALNOŚĆ

WADY

- > Nieużyteczna w systemach nieinteraktywnych (np. systemy wbudowane) > Subiektywność w ocenie
- > Brak dokładności przy nowych technologiach
- > Nie ocenia jakości kodu i złożoności
- > Czasochłonna i pracochłonna
- > Nieprzystosowana do metodyk zwinnych



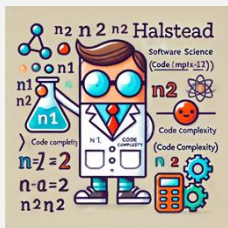
- **Metryki produktu: ZŁOŻONOŚĆ**

01 Nauka o programach Halsteada

Halstead's Software Science

02 Liczba cyklotematyczna McCabe'a

McCabe's cyclomatic number



Metryki produktu: **ZŁOŻONOŚĆ**

01 Nauka o programach Halsteada

Halstead's Software Science

> Halstead(1977) : Elements of Software Science, New York,

Elsevier North-Holland

> Bazuje na elementach składniowych programu

operatorach: "+", "*", "[...]", ";", ...

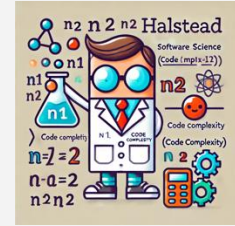
operandach: stałe, zmienne, wyrażenia

n_1 - liczba różnych operatorów

N_1 - całkowita liczba wystąpień operatorów

n_2 - liczba różnych operandów

N_2 - całkowita liczba wystąpień operandów



Metryki produktu: **ZŁOŻONOŚĆ**

01 Nauka o programach Halsteada

Halstead's Software Science

```
if (k < 2) {  
    if (k > 3)  
        x = x*k;  
}
```

operators:

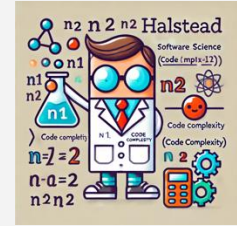
operands:

n_1 - liczba różnych operatorów

n_2 - liczba różnych operandów

N_1 - całkowita liczba wystąpień operatorów

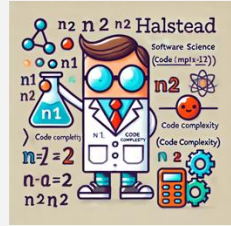
N_2 - całkowita liczba wystąpień operandów



Metryki produktu: **ZŁOŻONOŚĆ**

01 Nauka o programach Halsteada

Halstead's Software Science



```
if (k < 2) {  
    if (k > 3)  
        x = x*k;  
}
```

operatory: `if ( ) { } > < = * ;`

operandsy: `k 2 3 x`

n_1 - liczba różnych operatorów

n_2 - liczba różnych operandów

N_1 - całkowita liczba wystąpień operatorów

N_2 - całkowita liczba wystąpień operandów

Halstead's Software Science

```
operator: if ( ) { } > < = * ;      n1 = 10
```

operandy: k 2 3 x $n_2 = 4$

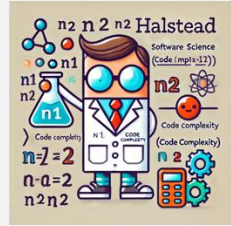
N_1 - całkowita liczba wystąpień operatorów

N₂- całkowita liczba wystąpień operandów

Metryki produktu: **ZŁOŻONOŚĆ**

01 Nauka o programach Halsteada

Halstead's Software Science



```
if (k < 2) {  
    if (k > 3)  
        x = x*k;  
}
```

operators: `if ( ) { } > < = * ;`

$n_1 = 10$ $N_1 = 13$

operands: `k 2 3 x`

$n_2 = 4$ $N_2 = 7$

n_1 - liczba różnych operatorów

N_1 - całkowita liczba wystąpień operatorów

n_2 - liczba różnych operandów

N_2 - całkowita liczba wystąpień operandów

MIARA	SYMBOL	FORMUŁA
DŁUGOŚĆ PROGRAMU	N	$N=N_1+N_2$
SŁOWNIK	n	$n=n_1+n_2$
VOLUME Liczba bitów konieczna do zakodowania słownika	V	$V=N\log_2 n$
EFFORT	E	$E=V*D$
STOPIEŃ TRUDNOŚCI Podatność na błędy	D	$D=\frac{n_1}{2} + \frac{N_2}{n_2}$
HALSTEAD PROGRAM LENGTH	H	$H=n_1\log_2 n_1+n_2\log_2 n_2$

ZALETY

- > Nie wymaga wnikliwej analizy kodu
- > Prognozuje stopień trudności utrzymania
- > Pomaga w planowaniu harmonogramu i raportowaniu
- > Mierzy ogólną jakość programu
- > Prosta do wyliczenia
- > Może być użyta dla dowolnego języka programowania
- > Doświadczalnie wykazano jej przydatność w predykcji wysiłku (programming effort) i oczekiwanej liczby błędów programistycznych

ZALETY

- > Nie wymaga wnikliwej analizy kodu
 - > Prognozuje stopień trudności utrzymania
 - > Pomaga w planowaniu harmonogramu i raportowaniu
 - > Mierzy ogólną jakość programu
 - > Prosta do wyliczenia
 - > Może być użyta dla dowolnego języka programowania
 - > Doświadczalnie wykazano jej przydatność w predykcji wysiłku (programming effort) i oczekiwanej liczby błędów programistycznych
- > Zależy od finalnej postaci kodu
 - > Nie nadaje się do wykorzystania na etapie projektowania (modelu)

WADY

- > Zależy od finalnej postaci kodu
- > Nie nadaje się do wykorzystania na etapie projektowania (modelu)

Metryki produktu: **ZŁOŻONOŚĆ**

02 Liczba cyklomatyczna McCabe'a

McCabe's cyclomatic number

- > Oparta na schemacie blokowym programu i liczbie niezależnych przebiegów
- > Przebiegi reprezentowane są za pomocą diagramu sterującego (grafu)

Węzły reprezentują sekwencyjne bloki kodu

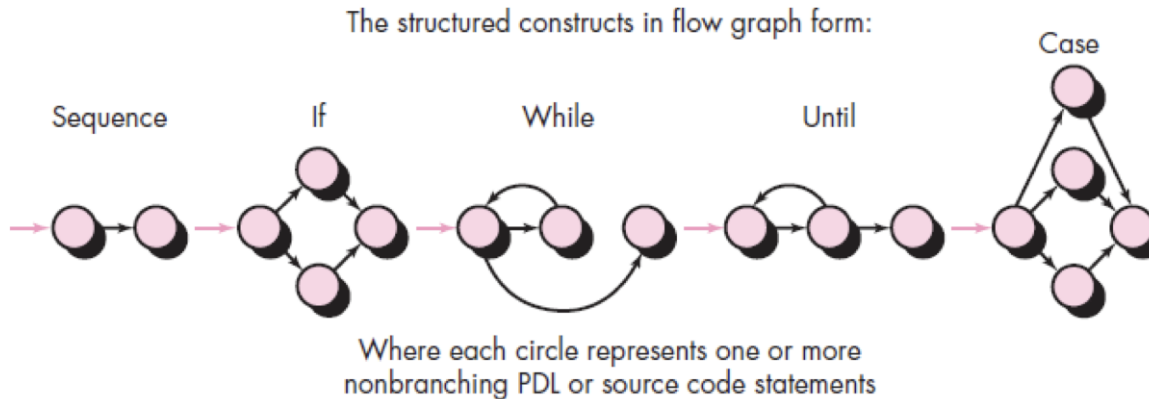
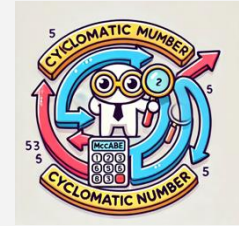
Krawędzie reprezentują możliwe ścieżki, zależne od decyzji



Metryki produktu: **ZŁOŻONOŚĆ**

02 Liczba cyklotematyczna McCabe'a

M McCabe's cyclomatic number

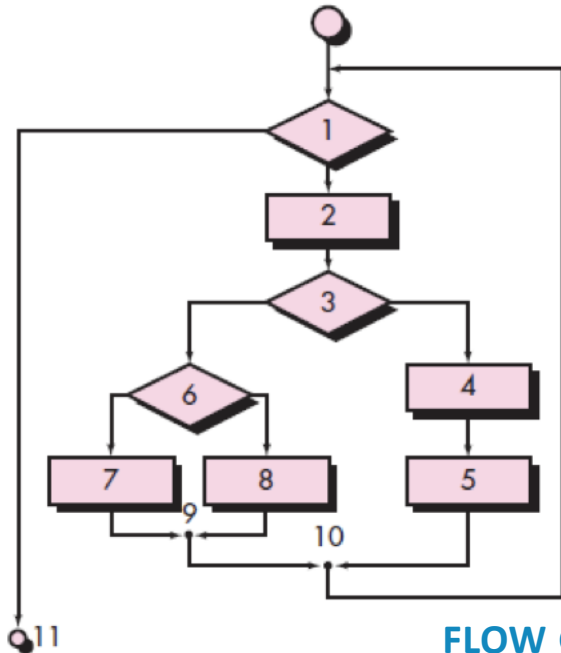
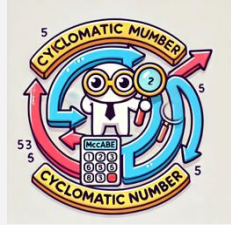


FLOW GRAPH NOTATION

Reference:- R.S. Pressman, Software Engineering: A Practitioner's Approach, McGraw-Hill, Ed 7, 2010.

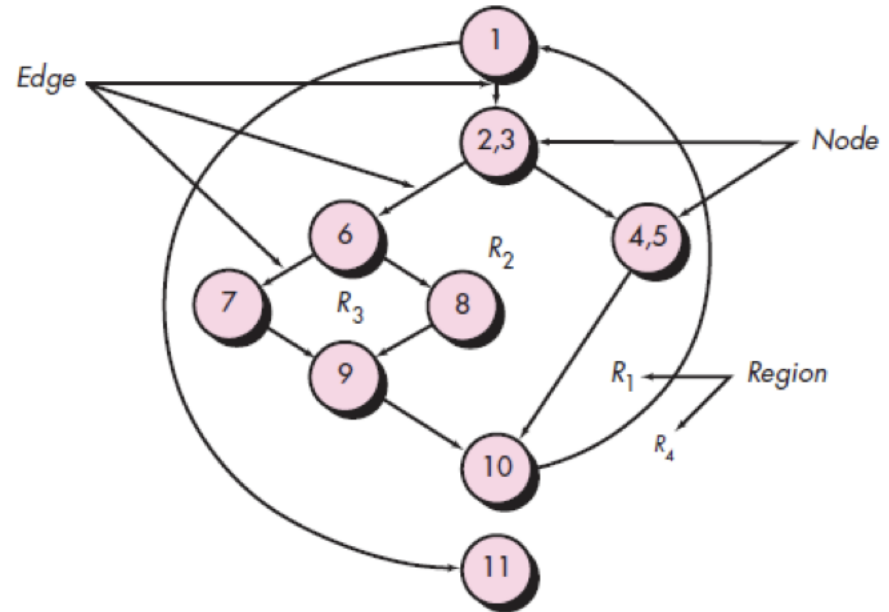
Metryki produktu: **ZŁOŻONOŚĆ**

02 Liczba cyklotyczna McCabe'a



(a)

FLOW GRAPH NOTATION



(b)

Metryki produktu: **ZŁOŻONOŚĆ**

02 Liczba cykloamatyczna McCabe'a

M McCabe's cyclomatic number

> Zbiór niezależnych ścieżek w grafie $V(G) = E - N + 2 = P + 1$

E liczba krawędzi

N liczba węzłów

P liczba węzłów warunkowych



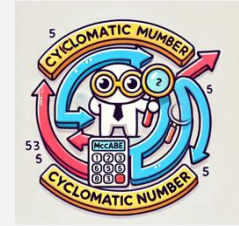
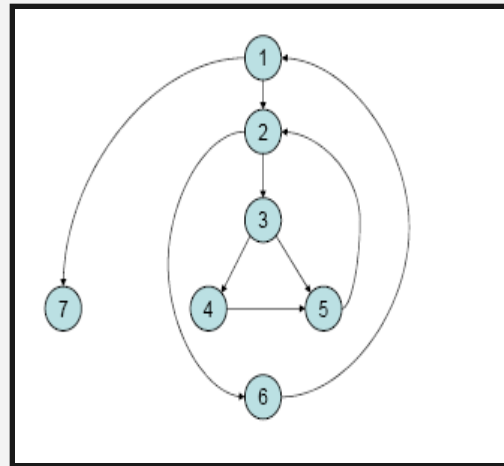
Metryki produktu: **ZŁOŻONOŚĆ**

02 Liczba cyklotyczna McCabe'a

M McCabe's cyclomatic number

$$V(G) = E - N + 2 = P + 1$$

```
i = 0;  
while (i < n-1) do  
  j = i + 1;  
  while (j < n) do  
    if A[i] < A[j] then  
      swap(A[i], A[j]);  
    end do;  
    i = i + 1;  
  end do;  
end do;
```



Metryki produktu: **ZŁOŻONOŚĆ**

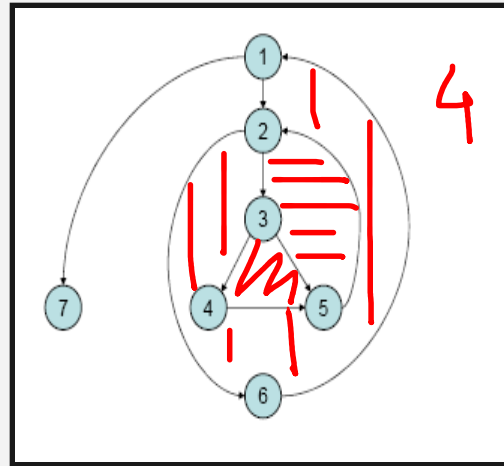
02 Liczba cyklotyczna McCabe'a

M McCabe's cyclomatic number



$$V(G) = E - N + 2 = P + 1$$

```
i = 0;
while (i < n-1) do
  j = i + 1;
  while (j < n) do
    if A[i] < A[j] then
      swap(A[i], A[j]);
    end do;
    i = i + 1;
  end do;
end do;
```



$$V(G) = 9 - 7 + 2 = 4$$

$$V(G) = 3 + 1 = 4$$

Zbiór przebiegów:

1, 7

1, 2, 6, 1, 7

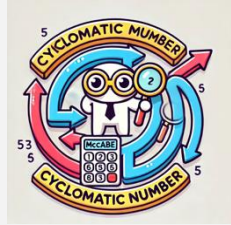
1, 2, 3, 4, 5, 2, 6, 1, 7

1, 2, 3, 5, 2, 6, 1, 7

Metryki produktu: **ZŁOŻONOŚĆ**

02 Liczba cyklomatyczna McCabe'a

M McCabe's cyclomatic number



INTERPRETACJA

- > $V(G)$ jest liczbą regionów wyznaczonych przez graf planarny
- > Liczba regionów wzrasta wraz z liczbą ścieżek warunkowych i pętli
- > Liczbowa miara trudności testowania
- > Doświadczenia pokazują, że $V(G)$ nie powinna przekraczać 10

Dla większej wartości testowanie jest bardzo trudne

Metryki produktu: **ZŁOŻONOŚĆ**

02 Liczba cykloamatyczna McCabe'a

McCabe's cyclomatic number

ZALETY

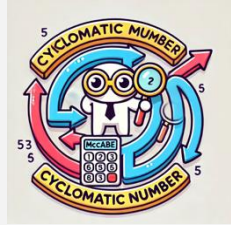
- > Do mierzenia stopnia trudności pielęgnacji
- > Miara jakości – porównanie różnych projektów
- > Może być wcześniej obliczona niż miara Halsteada
- > Prosta do wyliczenia



Metryki produktu: **ZŁOŻONOŚĆ**

02 Liczba cykloamatyczna McCabe'a

McCabe's cyclomatic number



WADY

- > Mierzy złożoność wątków kontroli, a nie złożoność danych
- > Pętle zagnieżdżone mają tą samą miarę, co nie zagnieżdżone
- > Zagnieżdżone struktury warunkowe są trudniejsze do zrozumienia
- > Może dawać mylne wyobrażenia dotyczących prostych porównań i struktur warunkowych

Metryki & obiekty

01 Lokalizacja

02 Hermetyzacja

03 Ukrywanie informacji

04 Dziedziczenie

05 Obiektowe techniki abstrakcyjne



Metryki & obiekty



DIT (Depth of Inheritance Tree)	Głębokość drzewa dziedziczenia obiektów
NOC (Number of Children)	Liczba potomków w ramach dziedziczenia dla konkretnej klasy
CBO (Coupling Between Object Classes)	Liczba obiektów, dla których analizowany obiekt jest łącznikiem do innych obiektów

Metryki & obiekty



RFC (Response for a Class)	Liczność zbioru metod (lokalnych i z innych obiektów) wywoływanych przez metody obiektu
LCOM (Lack of Cohesion in Methods)	Liczba metod nie wykorzystywanych przez inne metody
WMC (Weighted methods per class)	Rozmiar metod w klasach: zarówno liczba metod, jak i ich złożoność (liczona np.. tradycyjnymi metodami dla programów nieobiektowych).

• Metryki & obiekty •



PCTPUB (Percent Public Data)	Procent danych publicznych; liczony jako stosunek danych publicznych względem danych prywatnych w obiektach
PUBDATA (Access to Public Data)	Dostępność danych publicznych; określany miarą liczby dostępów do danych publicznych

Metryki & obiekty



PCTCALL (Percent of Unoverloaded Calls)	Stosunek liczby nieprzeciążonych wywołań metod do wszystkich wywołań
ROOTCNT (Number of Roots)	Globalna liczba korzeni w hierarchii dziedziczenia klas
FANIN (Fan-in)	Liczba klas, z których wywodzi się analizowana klasa

Metryki & obiekty



MAXV (Maximum $V(G)$)	Maksymalna wartość złożoności (cyklomatycznej) dla metod
MAXEV (Maximum $ev(G)$)	Maksymalna wartość złożoności struktury kodu dla metod
QUAL (Hierarchy Quality)	Liczba klas uzależnionych od poprawnego funkcjonowania swoich przodków

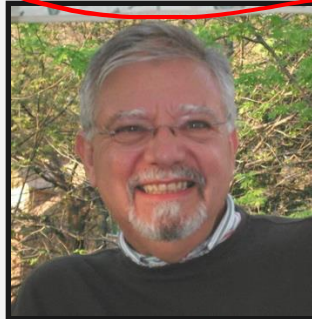
Jak i co mierzymy? Plan!



Metryki i gromadzenie danych

Jak się do tego zabrać?

Goal Question Metric

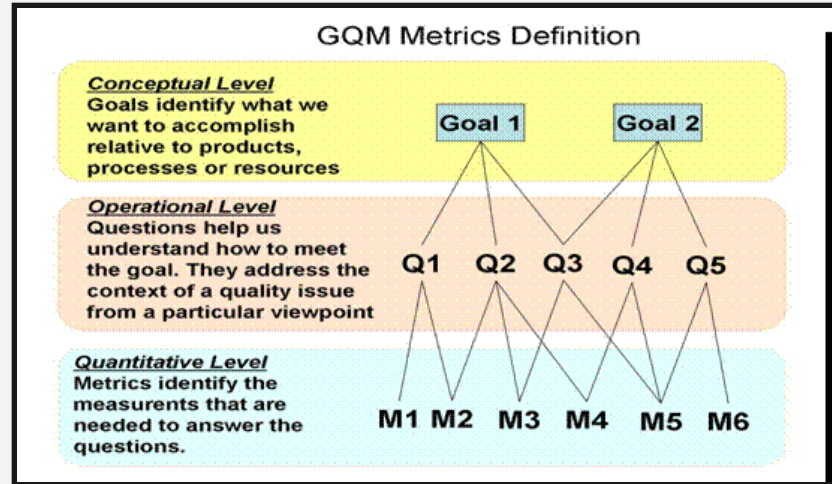


Victor R. Basili, Maryland University,
1988

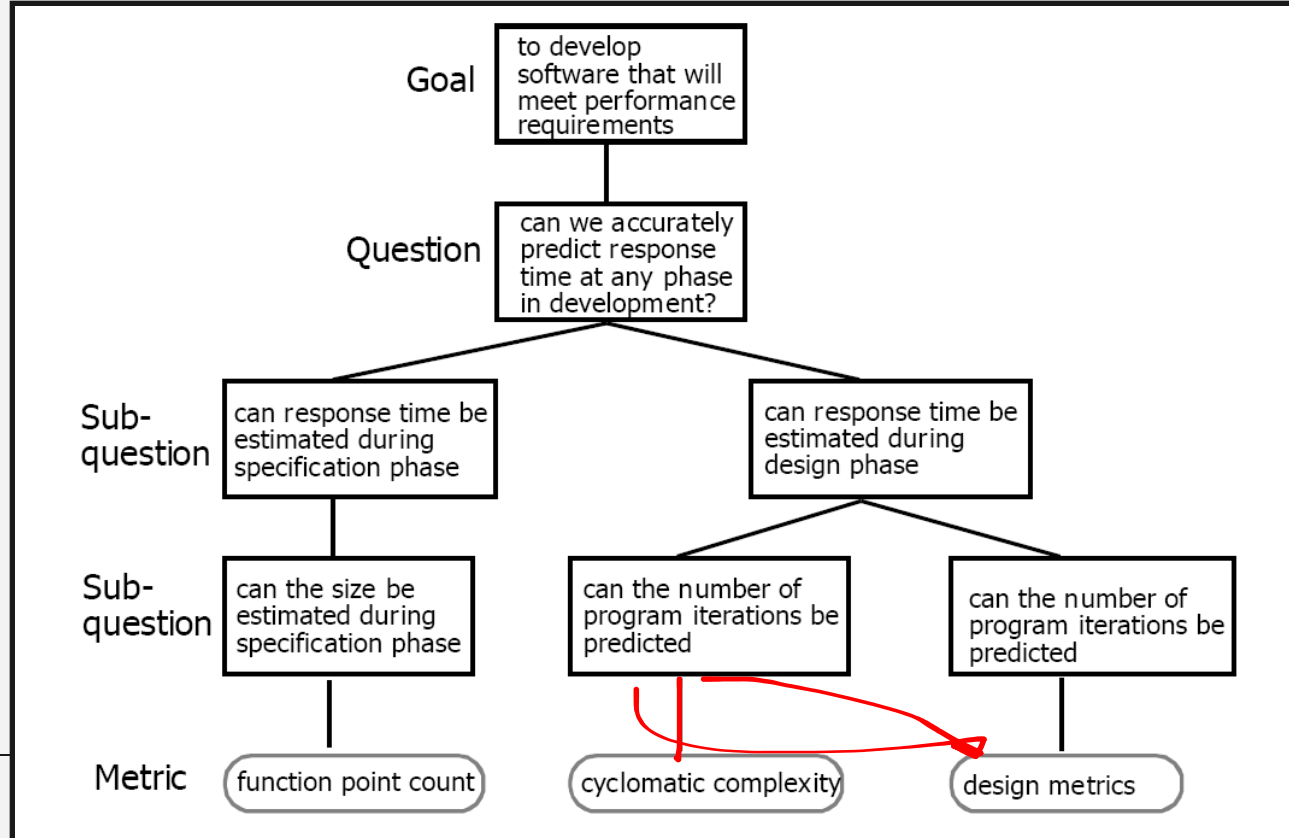
NASA Software Engineering
Laboratory (SEL)

Goal Question Metric GQM

- 01 Generate a set of goals
- 02 Derive a set of questions
- 03 Develop a set of metrics



Goal Question Metric GQM



Jak się do tego zabrać?



Konieczne do pomiarów:

- gromadzenie danych
- gotowe narzędzia

Jak się do tego zabrać?



Narzędzia do mierzenia rozmiaru kodu:

CLOC

Narzędzia do mierzenia funkcjonalności kodu:

SLIM-Suite of Tools

Narzędzia do mierzenia złożoności kodu:

SonarQube

Narzędzia do wizualizacji złożoności:

CodeCity

Etykieta pomiarów

Robert Grady



- 01 Interpretując wyniki, kieruj się zdrowym rozsądkiem
- 02 Udostępnij wyniki pracownikom i zespołom, którzy je prowadzą
- 03 Nie oceniaj poszczególnych pracowników na podstawie wyników pomiarów
- 04 Wspólnie z pracownikami zastanów się, po co pomiary i dobierz metryki do celów
- 05 Nigdy nie używaj wyników pomiarów, aby grozić pracownikom
- 06 Wyniki wskazujące na pojawiające się trudności nie są ZŁE – traktuj je jako wskazówkę, co można poprawić
- 07 Nie przywiązuj nadmiernej uwagi do jednej miary, zaniedbując inne

Uwagi M. Trzaska

01 Stosowanie tylko jednej metryki jest mało przydatne

02 Najlepszy zestaw metryk nie musi być od razu znany

dlatego na początku można stosować nawet kilkadziesiąt różnych technik

03 Docelowo najlepiej korzystać z 3 - 5 dobrze przemyślanych metryk

04 Metryki są prawie zawsze ze sobą powiązane i stosowanie jednej implikuje użycie drugiej

05 Pomiary nie mogą wpływać na mierzony element

06 Metryki mogą być szkodliwe tylko gdy je źle stosujemy



Uwagi

<http://www.elsop.com/wrc/humor/softmetr.htm>

The Pizza Metric

How: Count the number of pizza boxes in the lab.

What: Measures the amount of schedule under-estimation. If people are spending enough after-hours time working on the project that they need to have meals delivered to the office, then there has obviously been a mis-estimation somewhere.



Uwagi

<http://www.elsop.com/wrc/humor/softmetr.htm>

The Beer Metric

How: Invite the team to a beer bash each Friday. Record the total bar bill.

What: Closely related to the Aspirin Metric, the Beer Metric measures the frustration level of the team. Among other things, this may indicate that the technical challenge is more difficult than anticipated.





Koniec!